Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Dept. of Electrical and Computer Engineering

Course Code: ECE-2112
Course Title: Digital Techniques Sessional

Lab Report 01

Implementation of Logic Gates Using Universal Gates,
Implementation of Full Adder, and Binary to BCD Converter

Date of Submission: 26 July, 2026

Submitted To:

Mst. Mazeda Noor Tasnim
Lecturer,
Dept. of ECE, RUET

Submitted By

Mehedi Rahman Mahi
Roll: 2410015
Reg. No: 1068
Session: 2024-2025

Task 01: Implementation of Logic Gates Using Universal Gates

1. Implementation of an OR Gate Using Only NOR Gates

Objective

Design and verify an OR gate using two NOR gates, demonstrating that NOR gates alone can replicate other basic logic functions.

Theory

A NOR gate is universal because every other gate can be built from it alone. Since Y = complement of (A+B) is already the complement of OR, feeding this signal back into a second NOR gate whose inputs are tied together inverts it again:

$$G_1 = \overline{A+B}, \qquad Y = \overline{G_1 + G_1} = \overline{G_1} = A + B$$

Shorting both inputs of a NOR gate to the same signal X forces the output to equal the complement of X, so the gate behaves purely as an inverter in that configuration.

Truth Table

A	B	G_1	Y = A + B
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

Circuit Diagram

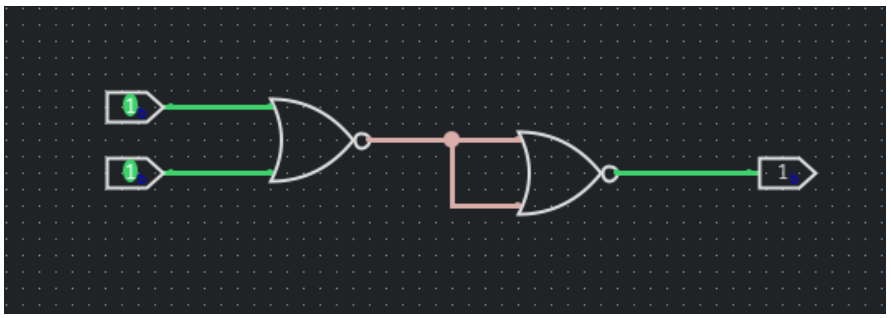

Figure 1: OR gate implemented using two NOR gates

Discussion

Tying both inputs of the second NOR gate to G_1 makes it act purely as an inverter, so the final output restores A + B. The truth table confirms the circuit matches the standard OR function across all input combinations.

Conclusion

An OR gate was successfully realized using two NOR gates, verified against its complete truth table.

2. Implementation of an OR Gate Using Only NAND Gates

Objective

Design and verify an OR gate using three NAND gates by applying De Morgan's theorem.

Theory

By De Morgan's theorem:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

which is precisely a NAND operation on the complements of A and B. Since a NAND gate with tied inputs behaves as an inverter, the complements of A and B are each generated by one NAND gate, and a third NAND gate combines them:

$$\overline{A} = \overline{A \cdot A}, \qquad \overline{B} = \overline{B \cdot B}, \qquad Y = \overline{\overline{A} \cdot \overline{B}} = A + B$$

Truth Table

A	B	Ā	B̄	Y = A + B
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Circuit Diagram

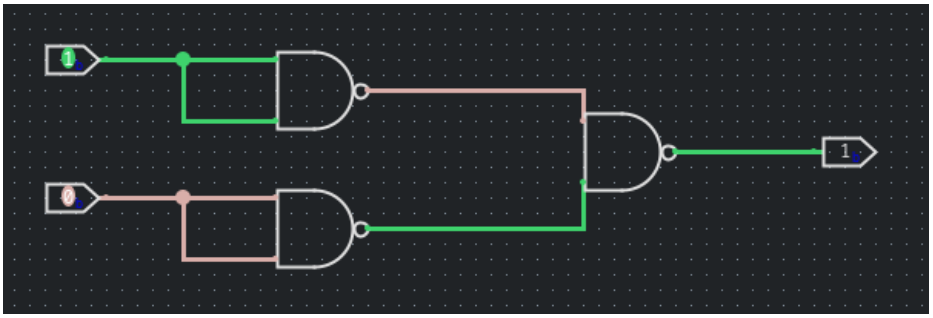

Figure 2: OR gate implemented using three NAND gates

Discussion

The first two NAND gates, with shorted inputs, act as inverters producing the complements of A and B. The third NAND gate combines these, which by De Morgan's theorem equals A + B. This needs one more gate than the NOR-based version, since NAND naturally produces AND-like combinations and requires an extra inversion stage to realize OR.

Conclusion

An OR gate was successfully realized using three NAND gates, matching the expected truth table for all input combinations.

3. Implementation of an AND Gate Using Only NOR Gates

Objective

Design and verify an AND gate using three NOR gates by applying De Morgan's theorem.

Theory

By De Morgan's theorem:

$$A \cdot B = \overline{\overline{A} + \overline{B}}$$

which is exactly a NOR operation on the complements of A and B. Using two NOR gates as inverters to generate these complements, a third NOR gate combines them:

$$\overline{A} = \overline{A + A}, \qquad \overline{B} = \overline{B + B}, \qquad Y = \overline{\overline{A} + \overline{B}} = A \cdot B$$

Truth Table

A	B	Ā	B̄	Y = A · B
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Circuit Diagram

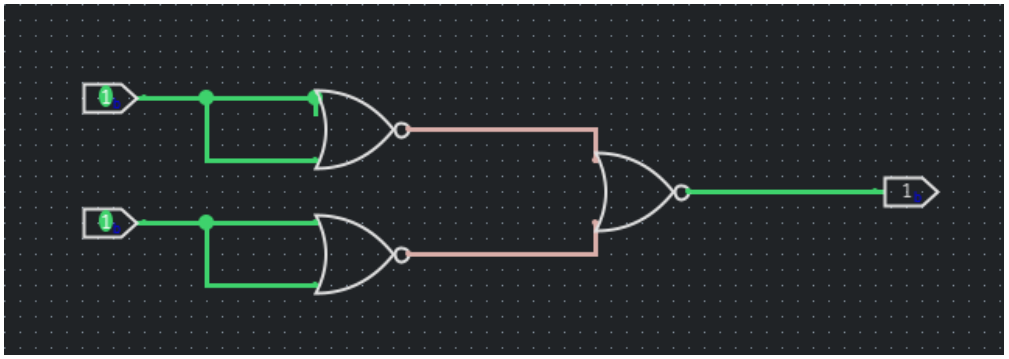

Figure 3: AND gate implemented using three NOR gates

Discussion

The first two NOR gates invert A and B to produce their complements; the third NOR gate combines these complements, giving the AND function by De Morgan's theorem. As with the NAND-based OR gate, this needs an extra inversion stage because NOR naturally produces OR-like combinations.

Conclusion

An AND gate was successfully realized using three NOR gates, with correct behaviour confirmed for all input combinations.

4. Implementation of an AND Gate Using Only NAND Gates

Objective

Design and verify an AND gate using two NAND gates.

Theory

A NAND gate is the complement of AND:

$$Y = \overline{A \cdot B}$$

Feeding this into a second NAND gate with tied inputs inverts it again, restoring the AND function:

$$G_1 = \overline{A \cdot B}, \qquad Y = \overline{G_1 \cdot G_1} = \overline{G_1} = A \cdot B$$

Truth Table

A	B	G_1 = complement of A·B	Y = A · B
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Circuit Diagram

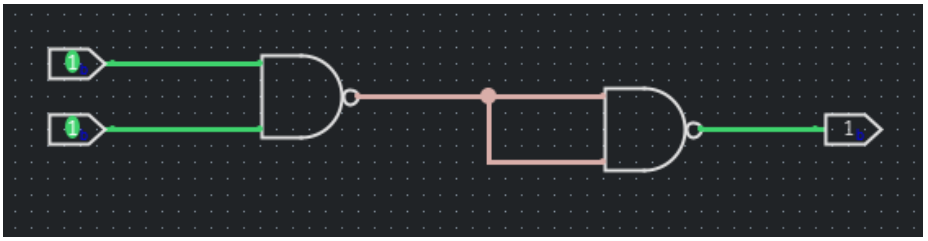

Figure 4: AND gate implemented using two NAND gates

Discussion

G_1 (the complement of A·B) is fed into both inputs of the second NAND gate, forcing it to invert once more and restoring the AND relationship. Since NAND is a universal gate, only two gates are needed, mirroring the NOR-based OR construction in Problem 1.

Conclusion

An AND gate was successfully realized using two NAND gates, matching the expected truth table.

5. Implementation of a NOT Gate Using Only a NOR Gate

Objective

Design and verify a NOT gate using a single NOR gate.

Theory

The NOR operation is:

$$Y = \overline{A + B}$$

Tying both inputs to the same signal A reduces this to:

$$Y = \overline{A + A} = \overline{A}$$

which is exactly the NOT function.

Truth Table

A	Y = Ā
0	1
1	0

Circuit Diagram

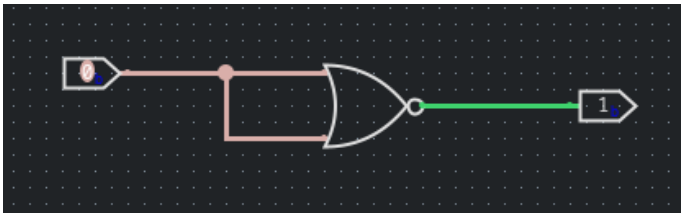

Figure 5: NOT gate implemented using a single NOR gate

Discussion

Shorting both NOR inputs removes the OR-combination behaviour, leaving only the inversion implicit in the NOR operation — the simplest possible universal-gate construction, and the same building block used in Problems 1 and 3.

Conclusion

A NOT gate was successfully realized using a single NOR gate with tied inputs.

6. Implementation of a NOT Gate Using Only a NAND Gate

Objective

Design and verify a NOT gate using a single NAND gate.

Theory

The NAND operation is:

$$Y = \overline{A \cdot B}$$

Tying both inputs to the same signal A reduces this to:

$$Y = \overline{A \cdot A} = \overline{A}$$

which is exactly the NOT function.

Truth Table

A	Y = Ā
0	1
1	0

Circuit Diagram

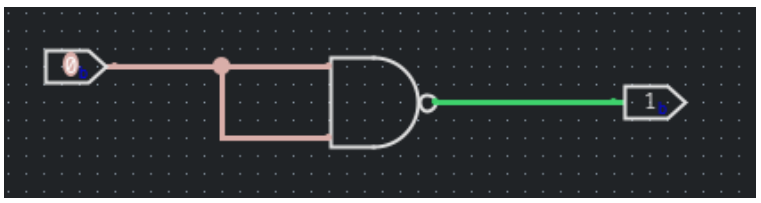

Figure 6: NOT gate implemented using a single NAND gate

Discussion

As with the NOR-based inverter, shorting both NAND inputs removes the AND-combination behaviour, leaving only inversion. This confirms that both NOR and NAND are, on their own, sufficient to build every other basic logic gate.

Conclusion

A NOT gate was successfully realized using a single NAND gate with tied inputs.

Task 02: Implementation of a Full Adder Circuit

Objective

Design a full adder from basic logic gates and verify its Sum and Carry outputs for all eight input combinations of A, B, and Cin.

Theory

A full adder adds three single-bit inputs — A, B, and carry-in Cin — producing a Sum bit and a carry-out bit Cout. Unlike a half adder, it accepts an incoming carry, allowing it to be cascaded into multi-bit adders:

$$Sum = A \oplus B \oplus C_{in}, \qquad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

It can be built from two half adders and an OR gate: the first computes P = A⊕B and A·B; the second combines P with Cin to give the final Sum and the term P·Cin; the OR gate then merges A·B and P·Cin to produce Cout.

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuit Diagram

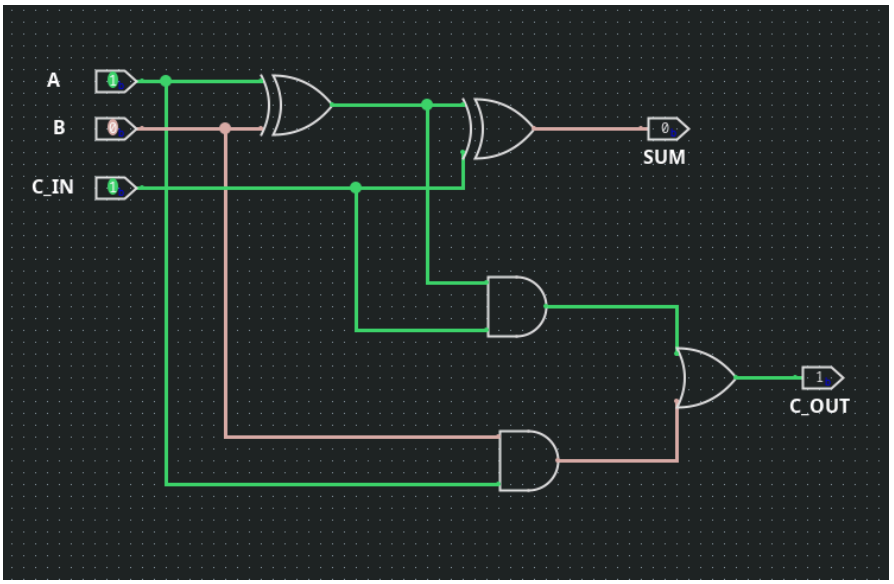

Figure 7: Full adder circuit using two XOR gates, two AND gates, and one OR gate

Discussion

The first XOR gate produces the partial sum P = A⊕B, while the first AND gate simultaneously produces the direct carry A·B. The second XOR gate combines P with Cin to give the final Sum, and the second AND gate produces P·Cin. The OR gate merges A·B and P·Cin into Cout, since a carry can be generated by either path. All eight rows of the truth table matched the expected outputs.

Conclusion

A full adder was successfully implemented and verified against its complete truth table, confirming correct three-bit binary addition.

Task 03: Implementation of a Binary to BCD Converter

Objective

Design a combinational circuit that converts a 4-bit binary number (0–15) into its two-digit Binary Coded Decimal (BCD) representation.

Theory

A 4-bit binary number B3B2B1B0 represents decimal values 0–15. Since BCD allows only digits 0–9, values from 10–15 need two digits: a tens digit T (0 or 1) and a units digit U3U2U1U0 (0–9). If the binary value is 9 or less, T = 0 and the units digit equals the input directly; if it is 10 or more, T = 1 and the units digit equals the input minus 10.
Karnaugh map simplification of the 16-row truth table gives:

$$T = B_3(B_2 + B_1)$$

$$U_3 = B_3\overline{B_2}\,\overline{B_1}$$

$$U_2 = \overline{B_2}B_3 + B_2\overline{B_1}$$

$$U_1 = B_1\overline{B_3} + \overline{B_2}B_1\overline{B_3}$$

$$U_0 = B_0$$

U0 always equals B0 directly, since subtracting 10 (an even number) never changes the parity of the least significant bit.

Truth Table

B3	B2	B1	B0	T	U3	U2	U1	U0
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1
0	0	1	0	0	0	0	1	0
0	0	1	1	0	0	0	1	1
0	1	0	0	0	0	1	0	0
0	1	0	1	0	0	1	0	1
0	1	1	0	0	0	1	1	0
0	1	1	1	0	0	1	1	1
1	0	0	0	0	1	0	0	0
1	0	0	1	0	1	0	0	1
1	0	1	0	1	0	0	0	0
1	0	1	1	1	0	0	0	1
1	1	0	0	1	0	0	1	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	1	0	0
1	1	1	1	1	0	1	0	1

Circuit Diagram

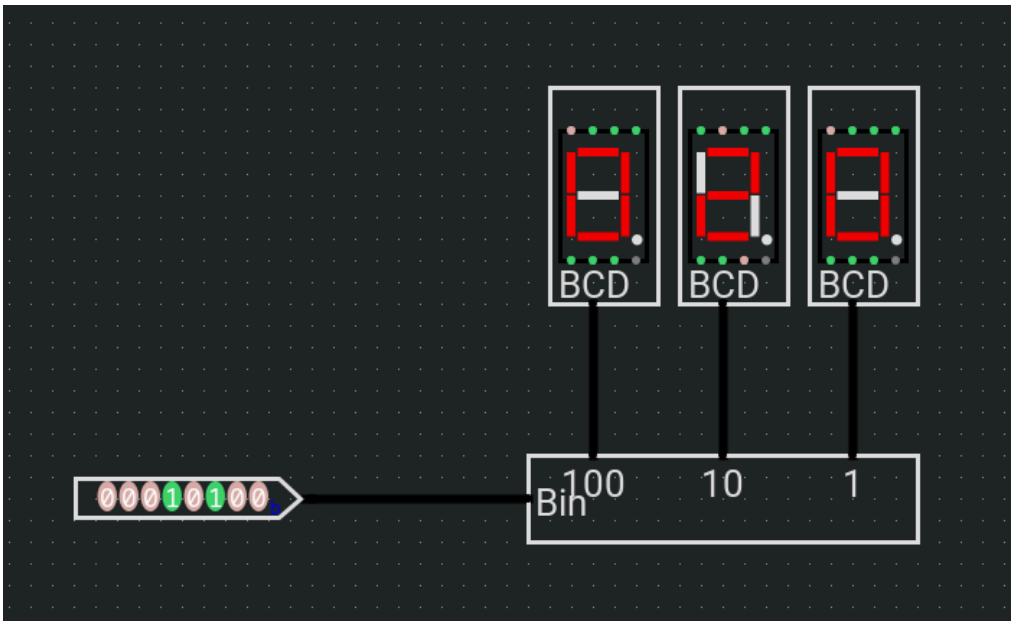

Figure 8: Binary to BCD converter driving three seven-segment BCD displays

Discussion

Each output is an independent Boolean function of B3B2B1B0, obtained from Karnaugh map simplification. The tens bit T acts as a threshold detector, going high only when the input reaches 10 or above. The units bits effectively subtract 10 whenever T = 1, which is why their expressions share terms with T. The whole block can be realized directly with AND, OR, and NOT gates, without a separate adder-subtractor stage, since the input range is restricted to 4 bits.

Conclusion

A binary to BCD converter for 4-bit inputs (0–15) was successfully designed using Karnaugh-map-derived combinational logic. The truth table confirmed correct separation into the tens bit T and units nibble U3U2U1U0 for every input value.

References

1. M. M. Mano, Digital Design, 5th ed. Upper Saddle River, N.J.: Pearson, 2013.
2. R. J. Tocci, N. S. Widmer, and G. L. Moss, Digital Systems: Principles and Applications, 11th ed. Upper Saddle River, N.J.: Pearson, 2011.